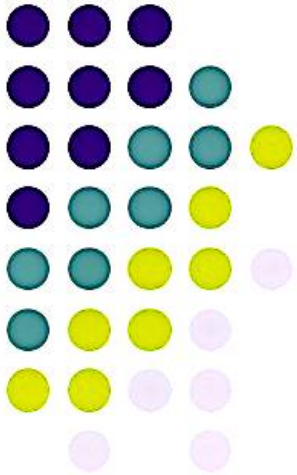
第11章
类的继承



扫描全能王 创建



学习目标

- 掌握基类和派生类的概念；
- 能够定义派生类，掌握不同继承方式下，派生类成员的访问控制；
- 掌握派生类的构造和析构；
- 理解虚基类的概念，能够利用虚基类解决共同基类导致的二义性问题；
- 掌握虚函数和多态性的概念，会定义和使用虚函数和纯虚函数。

扫描全能王 创建

11.1.1 基类与派生类

- 继承形成一个层次结构,一个类可由其它类通过继承得到,继承得到的新类称派生类(子类),被继承的称为父类（基类）。
- 单继承：只有一个基类的继承
- 多继承：有两个以上基类的继承

扫描全能王 创建

单选题 1分

关于派生类与基类之间关系，下面哪一种说法是错误的?

A 基类表示比较抽象的、一般性的、较大范畴的对象，派生类表示较具体的、特殊的、较小范畴的对象。

B 派生类的一个对象也是其基类的一个对象，这是无条件的。

C 派生类创建一个对象时，该类的所有直接或间接的基类也要实例化。

D 越具体的派生类对象包含越少的属性。

扫描全能王 创建

单选题 1分



对基类和派生类的关系描述中,错误的是()

A 派生类是基类的具体化

B 基类继承了派生类的属性

C 派生类是基类定义的延续

D 派生类是基类的特殊化

扫描全能王 创建

11.1.2 派生类的定义与构成

- 派生类定义格式：

```
class 派生类名:<继承方式1>基类名1,
               <继承方式2>基类名2,...,
               <继承方式n>基类名n
{
  成员说明；
};
```

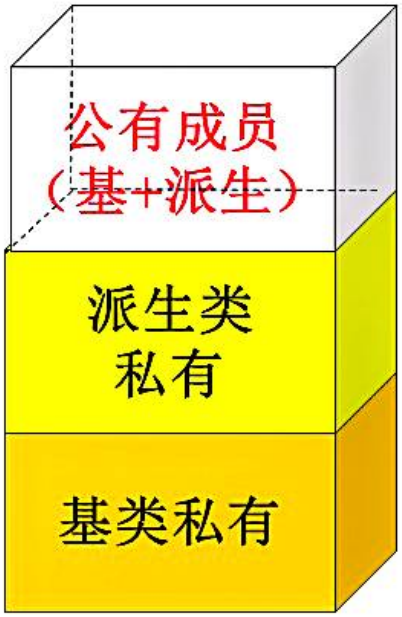
- 单一继承：只有一个基类名；多重继承：多个基类名
- access 为继承方式，可为private， public，protected或空,空时的含义是private。
- protected派生一般不使用

扫描全能王 创建

11.1.3 继承方式与访问控制

- **public派生：** 基类的公有成员如同是派生类的公有成员，即在派生类的外部也可访问基类的公有成员。基类中的私有成员仍是私有，即在基类外不能访问。





扫描全能王 创建

11.1.3 继承方式与访问控制

- **private派生：** 基类中的公有成员相当于在派生类中的私有成员.即在派生类中可以访问基类的公有成员,而在派生类外,不能访问基类的公有成员.



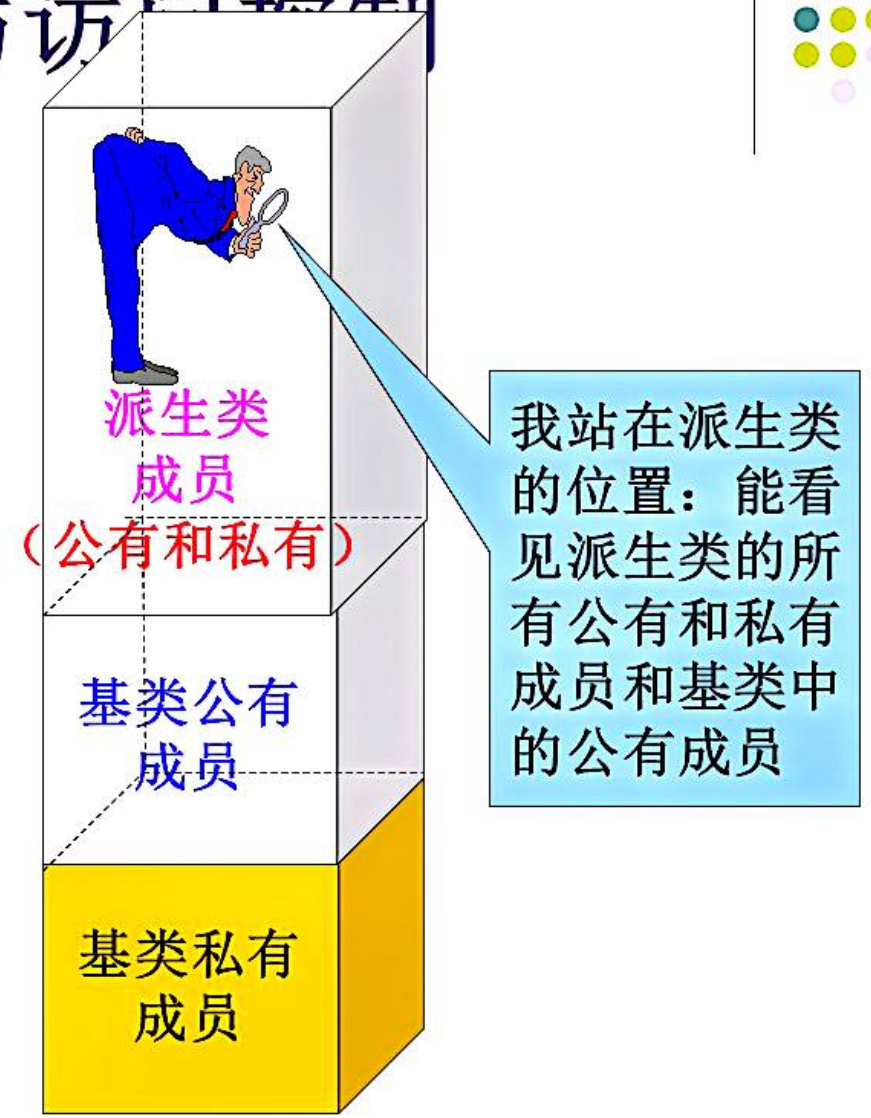
我能看见和使用所有的派生类中的公有成员基类公有成员在派生类外看，变为派生类的私有成员了



扫描全能王 创建

11.1.3 继承方式与访问控制

- 所有公有、私有派生：在派生类中可以访问和使用派生类中所有成员及基类中的公有成员。

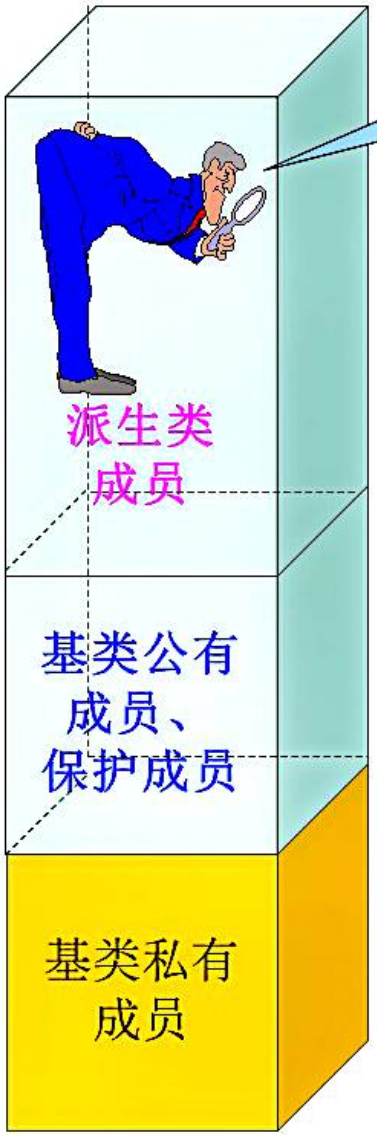


扫描全能王 创建

保护成员：



我站在基类外、派生类外，怎么也看不见保护成员



我站在派生类的位置，能看见和使用基类所有的保护成员、公有成员。

保护成员在基类外、派生类外都是不可见的，仅在派生类中可以使用，也就是，对派生类，保护成员就象公有的一样。

目的：使保护成员具有隐蔽性，但为了在派生类中提高操作效率而使用。

因此：若一个成员需要有私有的特性而又希望在派生类中不要通过成员函数接口直接使用，可以利用保护成员的特点。

扫描全能王 创建

```
#include <iostream>
#include <cstring>
using namespace std;

class Student{
protected:
  long  No;
 public:
  Student(long no){No=no;}
  long GetNo(){return No;}
};
```

```
class Emploee{
    char *Depart;
    char *Job;
public:
    Emploee(char *depart,char *job)
      {Depart=new char [strlen(depart)+1];
       strcpy(Depart,depart);
       Job = new char [strlen(job)+1];
       strcpy(Job,job);}
   ~Emploee()
     { if (Depart) delete  [ ] Depart;
       if (Job) delete  [ ] Job; }
  char *GetDepart(){return Depart;}
  char * GetJob(){return Job;}
};
```

```
class Student_Emploee:public Student,public Emploee {
public:
   Student_Emploee(char *depart,char *job,long n) :Student(n),Emploee(depart,job){}
   void Show() {cout<<"部门：  "<<GetDepart()<<'\t'<<"职务：  "<<GetJob()<<'\n';
                cout<<"学号：  "<<No<<'\n';}
};
int main()
{Student_Emploee a("computer","teacher",99062201);
 a.Show();}
```

若No为private，则需改为GetNo()

扫描全能王 创建

单选题 1分

派生类的成员函数可以直接访问基类的( )成员。

A 所有

B 公有和保护

C 保护和私有

D 私有

扫描全能王 创建

单选题 1分

对于公有继承,基类中的私有成员在派生类中将( )。

A 能够直接使用成员名访问

B 能够通过成员运算符访问

C 仍然是基类的私有成员

D 变为派生类的私有成员

扫描全能王 创建

单选题 1分

若有以下类定义:

```
class B{
      void fun1(){}
protected:
      double varl;
public:
      void fun2(){}
};
class D:public B{
protected:
      void fun3(){}
};
```

已知obj是类D的对象,下列语句中不违反类成员访问控制权限的是(　　)。

A obj.fun1();

B obj.varl;

C obj.fun2();

D obj.fun3();

扫描全能王 创建

单选题 1分

下面代码中哪一行代码编译出错?

```
class B {
protected:
    int x = 2;
};
class D : public B {
public:
    void f1() { x = 3; } //A
    void f1(D &rd) { rd.x = 4; } //B
    void f1(D *pd) { pd->x = 5; } //C
    void f1(B &rb) { rb.x = 6; } //D
};
```

基类中的保护成员是专门提供给自己的“同类”访问的。这里“同类”指的是具有同一个基类(它自己)的一组直接或间接的派生类。

A A

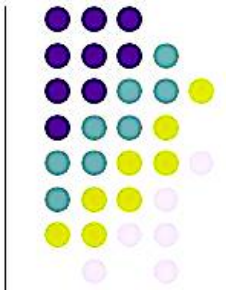
B B

C C

D D

扫描全能王 创建

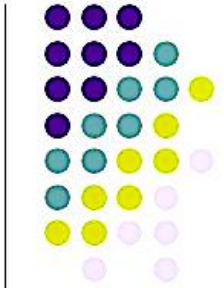
11.2 派生类的构造和析构



- 在基类中定义了基类的构造函数，并且在派生类中也定义了派生类的构造函数，那么，在产生派生类的对象时，一方面系统要调用派生类的构造函数来初始化在派生类中新增加的数据成员，另一方面系统要调用基类的构造函数来初始化派生类中的基类成员。这种调用基类的构造函数是由派生类的构造函数来确定的。

扫描全能王 创建

11.2 派生类的构造和析构

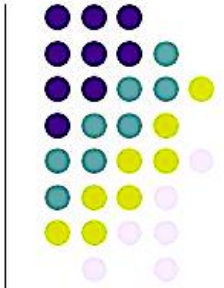


- 为了初始化基类成员，派生类的构造函数的一般格式为：

```
ClassName::ClassName(args):Base1(args1),Base2(args2),...,Basen(argsn)
{
    ... //初始化派生类中的其它数据成员
}
```

扫描全能王 创建

11.2 派生类的构造和析构



- 当说明派生类的对象时，系统首先调用各基类的构造函数(按说明顺序），对基类成员进行初始化，然后执行派生类的构造函数。如果某一个基类仍是派生类，则这种调用基类的构造函数的过程递归进行下去。当撤消派生类的对象时，析构函数的调用顺序正好与构造函数的顺序相反。

扫描全能王 创建

```
#include <iostream>
using namespace std;
class Base1 {
    int x;
public:
  Base1(int a=30)
   {   x=a;   cout<<"调用基类1的构造函数!"<<endl; }
 ~Base1( ) {  cout<<"调用基类1的析构函数!"<<endl; }
};

class Base2 {
   int y;
public:
  Base2(int a)
   {  y=a;   cout<<"调用基类2的构造函数!"<<endl; }
 ~Base2( ) { cout<<"调用基类2的析构函数!"<<endl;}
};

class Derived:public Base1, public Base2{
   int z;
public:
    Derived(int a,int b):Base1(a),Base2(20)
      { z=b;      cout<<"调用派生类的构造函数!" <<endl;}
   ~Derived( ){  cout<<"调用派生类的析构函数!"<<endl; }
};
```

1)int a=30改为int a
2)Base1(a)删去-出错

删去,Base2(20)

```
int main(void)
{
 Derived  c(100,200);
}
```

输出结果：

```
调用基类1的构造函数!
调用基类2的构造函数!
调用派生类的构造函数!
调用派生类的析构函数!
调用基类2的析构函数!
调用基类1的析构函数!
```

扫描全能王 创建

单选题 1分

建立包含有类对象成员的派生类对象时,自动调用构造函数的执行顺序依次为( )的构造函数。

A 自己所属类、对象成员所属类、基类

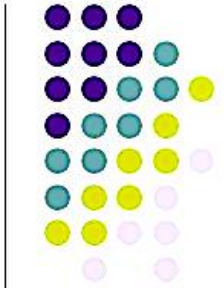
B 对象成员所属类、基类、自己所属类

C 基类、对象成员所属类、自己所属类

D 基类、自己所属类、对象成员所属类

扫描全能王 创建

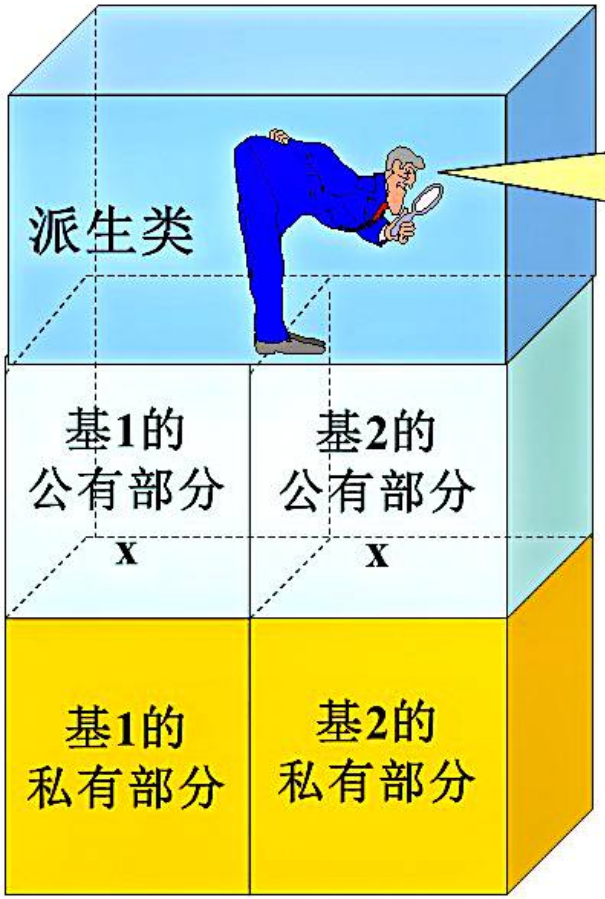
11.3 二义性问题



- 若一个公有的派生类是由二个或多个基类所派生，当基类中成员的访问权限为**public**，且不同基类中的成员具有相同的名字时，出现了重名的情况。这时在派生类使用到基类中的同名的成员时，出现了不唯一性，这种情况称为冲突。

扫描全能王 创建

11.3 二义性问题



现在能看到两个x，你要给出一个方法使我能区分两个来自不同基类的x

解决：

1）改名

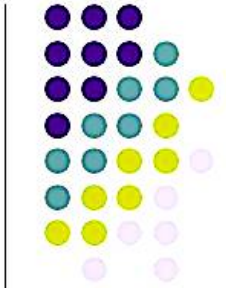
2）改访问权限为私有

3）用类名：：成员名限定

(这种关系只能一层）

扫描全能王 创建

11.3 二义性问题



- 在C++中，允许派生类中新增加的成员名与其基类的成员名相同，这种同名并不产生冲突。当没有使用作用域运算符时，则派生类中定义的成员名优先于基类中的成员名，这种优先关系称为支配规则。

扫描全能王 创建

11.3 二义性问题

```
#include <iostream>
using namespace std;

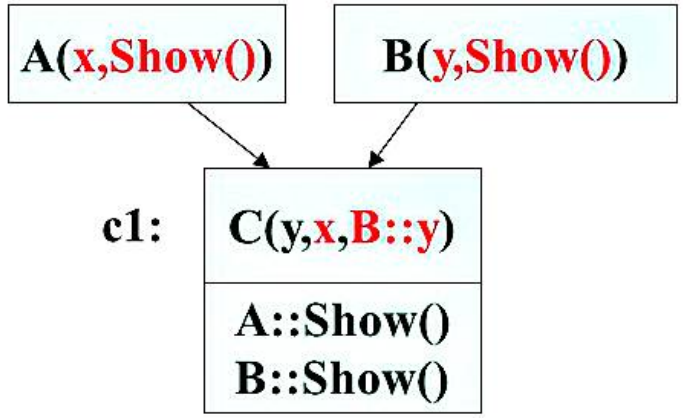
class A{
 public:
    int x;
   void Show(){cout<<"x="<<x<<'\n';}
};

class B{
 public:
  int y;
  void Show(){cout<<"y="<<y<<'\n';}
};

class C:public A,public B{
public:
   int y;
};
```

注意：基类对象不能多次继承

```
int main()
{
  C c1;
  c1.x=100;
  c1.y=200; //给派生类中的y赋值
  c1.B::y=300; //给基类B中的y赋值
  c1.A::Show();
  c1.B::Show();
    //用作用域运算符限定调用的函数
  cout<<"y="<<c1.y<<'\n';
    //输出派生类中的y值
  cout<<"y="<<c1.B::y<<'\n';
   //输出基类B中的y值
}
```



扫描全能王 创建

单选题 1分

当派生类中有和基类一样名字的成员时,一般来说,( )。

A 将产生二义性

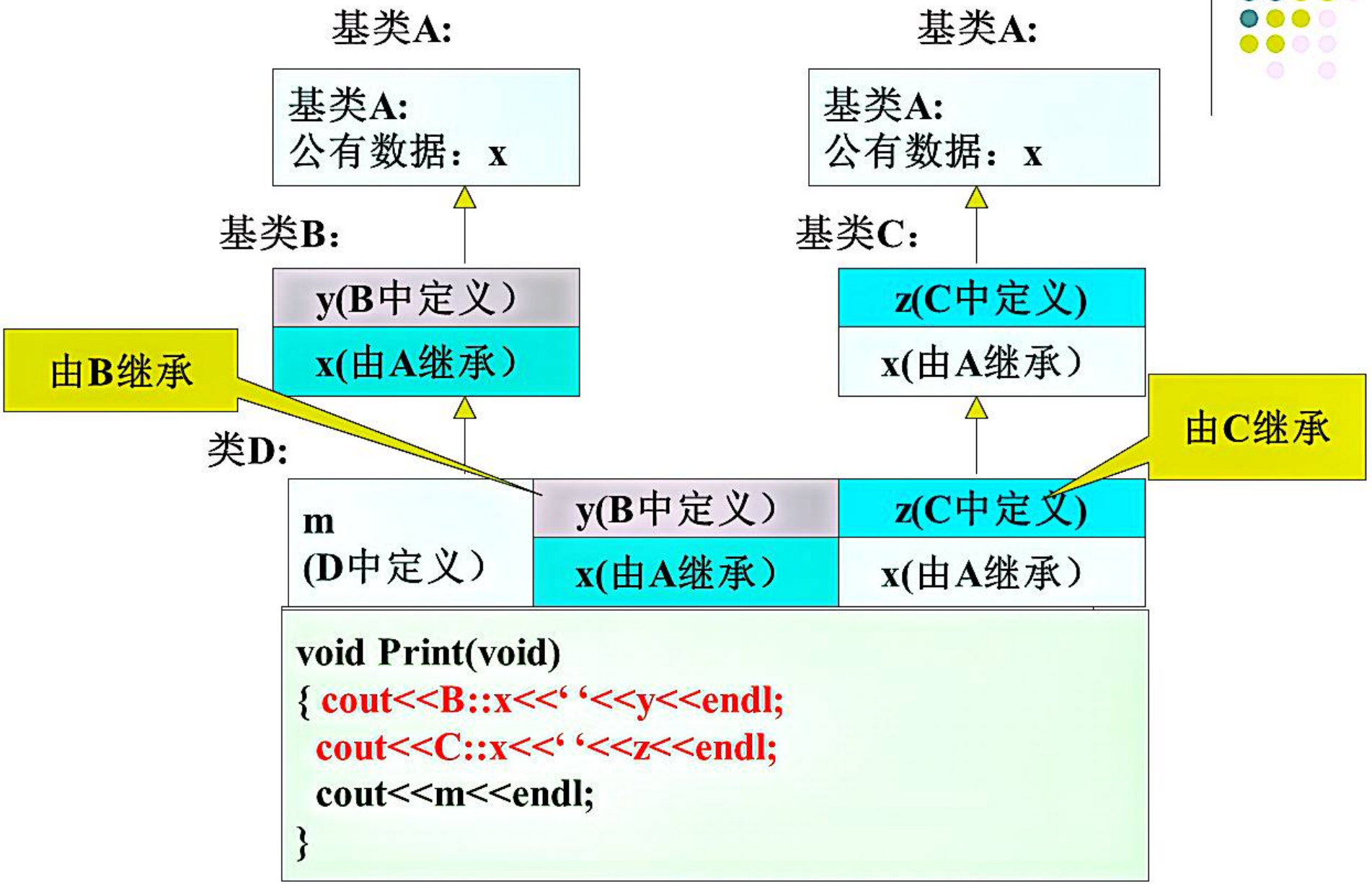
B 派生类的同名成员将覆盖基类的成员

C 是不能允许的

D 基类的同名成员将覆盖派生类的成员

扫描全能王 创建

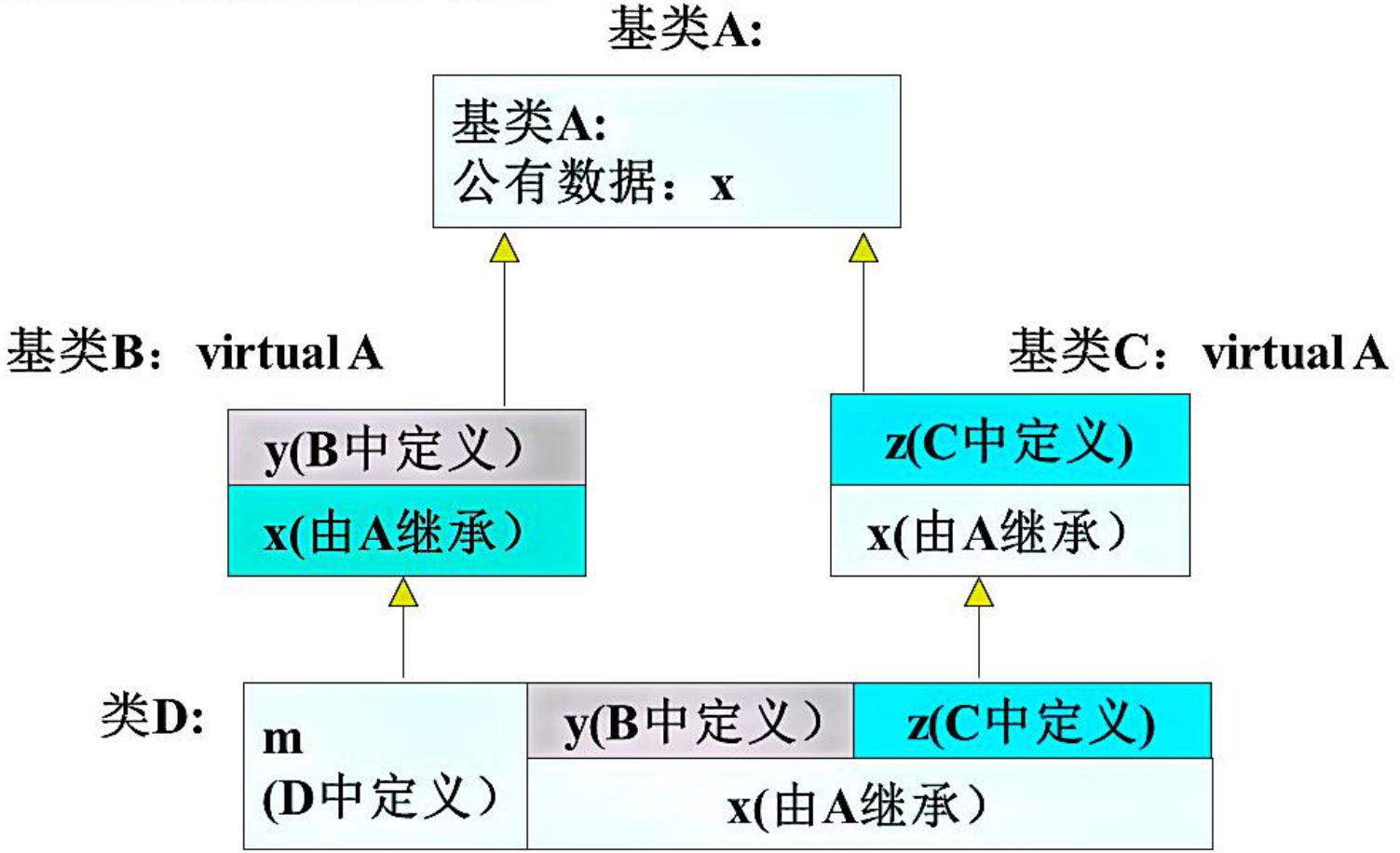
11.4 虚基类



扫描全能王 创建

11.4 虚基类

解决继承中的重复问题



扫描全能王 创建

11.4 虚基类

- 若有虚基类，那么构造函数的调用顺序为：先按虚基类说明的顺序调用虚基类的构造函数，再按非虚基类（若有的话）说明顺序调用相应的构造函数，最后再调用此派生类的构造函数。虚基类的构造函数仅仅做一次。

扫描全能王 创建

11.4 虚基类

```
#include <iostream>
using namespace std;

class A{
 public:
   int x;
   A(int a) { x=a;}};

class B:virtual public A{
 public:
    int y;
    B(int a, int b ):A(b) { y=a;}
    void PB()
     {cout<<"x="<<x<<'\t'<<"y="<<y<<'\n';}
};

class C:public virtual A{
 public:
   int z;
   C(int a,int b):A(b) { z=a;}
    void PC()
      { cout<<"x="<<x<<'\t'<<"z="<<z<<'\n'; }
};

class D: public B,public C {
public:
   int m;
   D(int a, int b,int d,int e,int f):B(a,b),C(d,e),A(20)
     { m=f; }
   void Print(void)
     { PB();
       PC();
       cout<<"m="<<m<<'\n';}
};
int main(void)
{
  D d1(100,200,300,400,500);
  d1.Print();
  d1.x=400;
  d1.Print();
}
```

执行以上程序，输出以下6行：

```
x=20        y=100
x=20        z=300
m=500
x=400       y=100
x=400       z=300
m=500
```

扫描全能王 创建

```
#include <iostream>
using namespace std;
class X{
 public:
    int x;
    X(int a){x=a;}
};
class A:virtual public X{
 public :
    A(int a):X(a){}
};
class B:virtual public X{
 public:
    B(int a):X(a){}
};

class C:public A,public B{
 public:
C(int a):A(20),B(30),X(40){}
};
```

若X(int a)改为：
X(int a=20)
则下面的,X(40)
可去掉

```
int main()
{
    C c(30);
    cout<<c.x<<endl;
}
```

A)20 B)30 C)40 D)50

,X(40)去掉，出错

扫描全能王 创建

单选题 1分

下面程序的输出结果是(　　)

```
#include <iostream>
using namespace std;
class Base
{ public:
    Base(int i) { cout << i; }
    ~Base () { }
};
class Base1: virtual public Base
{ public:
    Base1(int i, int j=0) : Base(j) { cout << i; }
    ~Base1() {}
};
class Base2: virtual public Base
{public:
    Base2(int i, int j=0) : Base(j) { cout << i; }
    ~Base2() {}
};
class Derived : public Base2, public Base1
{
  public:
    Derived(int a, int b, int c, int d) : mem1(a), mem2(b), Base1(c),  Base2(d), Base(a)
   { cout << b; }
  private:
    Base2 mem2;
    Base1 mem1;
};
int main() { Derived objD (1, 2, 3, 4); return 0; }
```

A 134122

B 123412

C 14302012

D 143212

扫描全能王 创建

11.5 子类型关系

派生类对象能自动地当作基类对象使用，它包含基类的所有成员；反之，不成立。

```
class B{ …..};
class D :public B
  {

  };
B b,*p;
D d;
```

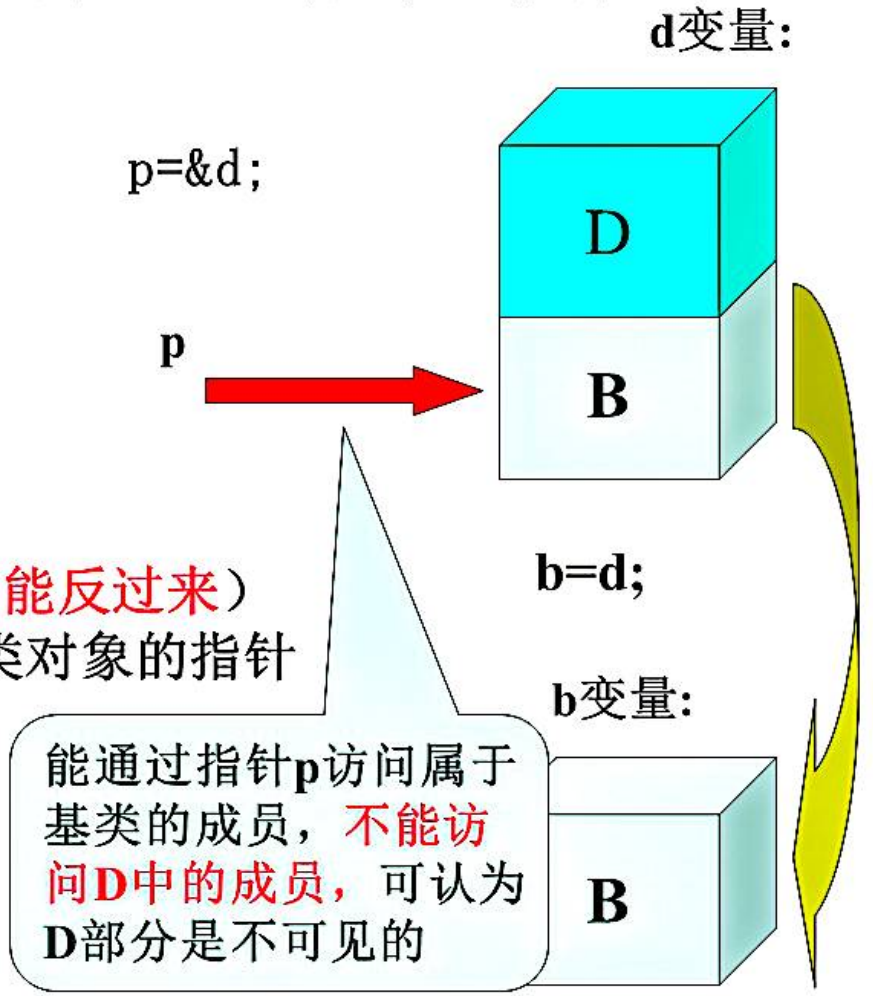
1)派生类对象向基类对象赋值 b=d;(不能反过来）

2)派生类对象的地址可以赋给指向基类对象的指针

p=&d;

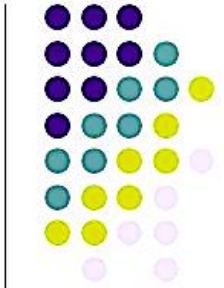
3)派生类对象可以初始化基类的引用

B &r=d;



扫描全能王 创建

11.6 虚函数



❖根据赋值兼容规则，对象可以作为它自己的类型或它的基类的对象使用，还能通过基类的地址被操作。取一个对象的地址（或指针或引用），并看作基类的地址，这被称为向上映射，因为继承树是以基类为顶点的。

扫描全能王 创建

11.6 虚函数

```
#include <iostream>
using namespace std;

class A{
  int x;
public:
   A(){x =1000;}
   void print()
   {cout <<"x="<<x<<'\t';}
};

class B:public A{
   int y;
public:
   B() { y=2000;}
   void print()
   {cout <<"y="<<y<<'\t';}
};

class C:public A{
   int z;
public:
   C(){z=3000;}
   void print()
   {cout <<"z="<<z<<'\n';}
};

int main( )
{
   A a, *pa;
   B b;
   C c;
   a.print();
   b.print();
   c.print();
   pa=new A;
   pa->print();
   delete pa;
   pa=new B;
   pa->print();
   delete pa;
   pa=new C;
   pa->print();
   delete pa;
}
```

执行该程序后输出以下2行：

```
x=1000        y=2000        z=3000
x=1000        x=1000        x=1000
```

扫描全能王 创建

```
#include <iostream>
using namespace std;
class A{
  int x;
public:
   A(){x =1000;}
   virtual void print()
   {cout <<"x="<<x<<'\t';}
};

class B:public A{
   int y;
public:
   B() { y=2000;}
   void print()
   {cout <<"y="<<y<<'\t';}
};

class C:public A{
   int z;
public:
   C(){z=3000;}
   void print()
   {cout <<"z="<<z<<'\n';}};
```

```
int main( )
{
   A a, *pa;
   B b;
   C c;
   a.print();
   b.print();
   c.print();
   pa=new A;
   pa->print();
   delete pa;
   pa=new B;
   pa->print();
   delete pa;
   pa=new C;
   pa->print();
   delete pa;
}
```

执行该程序后输出以下2行：

x=1000 y=2000 z=3000

x=1000 y=2000 z=3000

将virtual void print() 改为 void print()，输出变为：

x=1000 y=2000 z=3000

x=1000 x=1000 x=1000

扫描全能王 创建

11.6 虚函数

- 多态的体现：
  - 函数的重载 - 编译时多态：编译时能确定用哪个重载函数。

```
int abs(int  i)                          float abs(float x)
 {return (i>0)?i:-i;}                     {return (x>0.0)?x:-x;}
  abs(-10);                               abs(10.0)
```

  - 虚函数 - 运行时的多态：编译时无法确定，只有在运行时才能确定。
- 定义：
  - 成员函数前加上**virtual**，就成为了虚函数。虚函数具有继承性，派生类中都具有该性质。因此在派生类中重新定义该函数功能时前面可加也可不加**virtual**。
  - 但在派生类中函数的形参（包括个数、类型）不能变，函数返回值类型不能变，否则，在派生类中该函数就没有虚的特性了
- 目的：实现一个名字（接口），不同实现 。

扫描全能王 创建

虚函数需要注意什么？

（1）给出了一个统一名字
（2）设置一个基类指针
（3）通过将基类指针指向派生类或基类变量后，通过指针访问虚函数而不要去顾及现在的对象是什么。

```
#include <iostream>
using namespace std;

class B{
public:
virtual void vf(){cout<<"Class B\n";}
};

class D:public B{
public:
 void vf(){cout<<"Class D\n";}
};

class E:public B{
public:
void vf(){cout<<"Class E\n";}
};
```

vf()
改成：
vf(int a=4)

D中的vf函数不再具有虚特性。
输出：
Class B
Class B
Class B

```
int main(){
  B b,*pb;
  D d;
  E e;
   pb=&b;
   pb->vf();
   pb=&d;
   pb->vf();
   pb=&e;
   pb->vf();
}
```

输出：
Class B
Class D
Class E

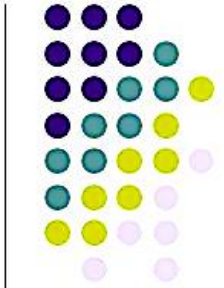
将B类中virtual 去掉

```
int main(){
  B b,*pb;
  D d;
  E e;
   pb=&b;
   pb->vf();
   pb=&d;
   pb->vf();
   pb=&e;
   pb->vf();
}
```

输出：
Class B
Class B
Class B

扫描全能王 创建

11.6.2 成员函数中调用虚函数



- 成员函数中可以直接调用本类中的虚函数。
- 虚函数是通过当前对象的指针this来调用的。

扫描全能王 创建

```
#include <iostream>
using namespace std;

class A{
public:
  virtual void fun1()
     {cout <<"A::fun1"<<'\t';
      fun2(); }
  void fun2()
     { cout<<"A::fun2"<<'\t';
       fun3(); }
  virtual void fun3()
     {cout <<"A::fun3"<<'\t';
      fun4();}
   virtual void fun4()
     {cout <<"A::fun4"<<'\t';
      fun5();}
   void fun5()
     {cout<<"A::fun5"<<'\n'; }
};

class B:public A{
public:
   void fun3()
    { cout <<"B::fun3"<<'\t';
      fun4(); }
   void fun4()
    { cout <<"B::fun4"<<'\t';
      fun5();}
   void fun5()
     { cout <<"B::fun5"<<'\n';}
};

int main(void )
{
   A a;
   a.fun1();
   B b;
   b.fun1();
}
```

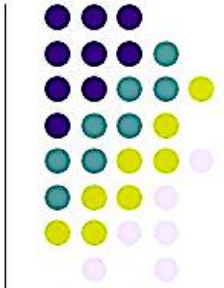
执行程序后输出以下2行：

A::fun1 A::fun2 A::fun3 A::fun4 A::fun5

A::fun1 A::fun2 B::fun3 B::fun4 B::fun5

扫描全能王 创建

11.6.3 构造函数中调用虚函数



- 构造函数中可以直接调用本类中的虚函数。
- 没有虚构造函数

扫描全能王 创建

```
#include <iostream>
using namespace std;

class A{
public:
    virtual void fun()
     {cout <<"A::fun"<<'\t';}
    A(){ fun(); }
};

class B:public A{
public:
    B() { fun(); }
    void fun(){ cout<<"B::fun"<<'\t'; }
   void g() { fun(); }
};

class C:public B{
public:
    C() { fun(); }
     void fun()
      { cout<<"C::fun"<<'\n';}
};

int main(void )
{
  C c;
   c.g();
}
```

执行程序后的输出为：

A::fun B::fun C::fun
C::fun

扫描全能王 创建

11.6.4 虚析构函数

- 析构函数可以是虚的，从带有虚析构函数的基类派生出的类的析构函数也是虚函数。此时允许一个指向基类的指针在它实际上指的是派生类对象时也能正确的调用。

扫描全能王 创建

11.6.4 虚析构函数

例：

```
#include <iostream>
using namespace std;
class color{
public: virtual ~color(){cout<<"color\n";}
};

class red:public color
{  public: ~red(){cout<<"red\n";}//虚析构函数
};

class brightred:public red
{ public:~brightred(){cout<<"brightred\n";} //虚析构函数
};

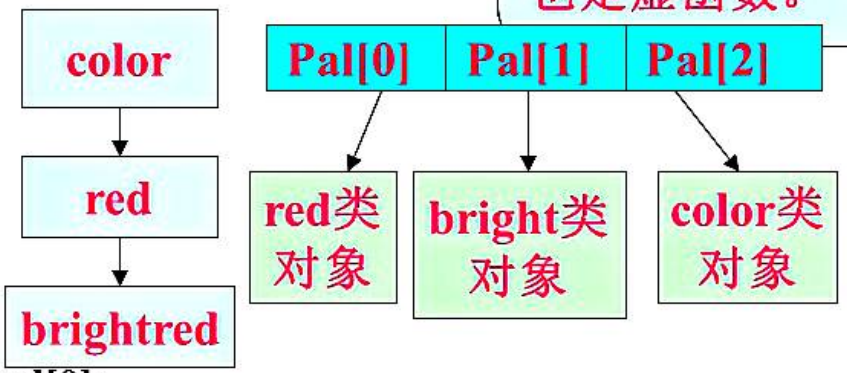
int main()
{ color *pal[3];
pal[0]=new red;
pal[1]= new brightred;
pal[2]=new color;
cout<<"pal[0] delete\n";   delete pal[0];
cout<<"pal[1] delete\n";   delete pal[1];
cout<<"pal[2] delete\n";   delete pal[2];
}
```

去掉virtual

输出：

```
pal[0] delete
red
color
pal[1] delete
brightred
red
color
pal[2] delete
color
```

~color的虚函数特性使得派生类的~red,~brightred也是虚函数。




```
pal[0] delete
color
pal[1] delete
color
pal[2] delete
color
```

扫描全能王 创建

单选题 1分

面向对象方法的多态性是指(　)。

A 一个类可以派生出多个特殊类

B 一个对象在不同的运行环境中可以有不同的变体

C 针对一消息,不同的对象可以以适合自身的方式加以响应

D 一个对象可以是由多个其他对象组合而成的

扫描全能王 创建

单选题 1分

实现运行时多态的机制是(　　)

A 虚函数

B 重载函数

C 静态函数

D 模版函数

扫描全能王 创建

单选题 1分

下面程序的输出结果是(　　)

```
#include <iostream>
using namespace std;
class Base
{
  public:
     virtual void f() { cout <<  "f0+" ; }
     void g() { cout <<  "g0+" ; }
};
class Derived : public Base
{
  public:
     void f() { cout <<  "f+" ; }
     void g() { cout <<  "g+" ; }
};
int main()
{ Derived d;
  Base *p = &d;
  p->f();  p->g();
  return 0; }
```

A　f+g+

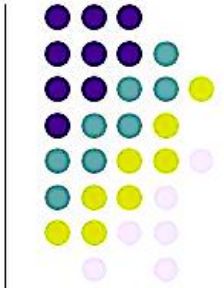
B　f0+g+

C　f+g0+

D　f0+g0+

扫描全能王 创建

11.6.5 纯虚函数与抽象类



- 纯虚函数：
  - 目的：为派生出的类提供统一的接口，在派生类中必须对纯虚函数定义。
  - 格式：**virtual <类型>函数名（参数表）= 0**
- 抽象类：
  - 至少含有一个纯虚函数的类，抽象类只能作为派生类的基类，不能用来说明这种类的对象。

扫描全能王 创建

11.6.5 纯虚函数与抽象类

```
#include <iostream>
using namespace std;

class A{
  int x;
public:
    A(){x =1000;}
   virtual void print()=0; //纯虚函数
}; //A是抽象类
class B:public A{
   int y;
public:
   B(){ y=2000;}
   void print(){cout <<"y="<<y<<'\n';}
};
```

执行该程序后的输出为：

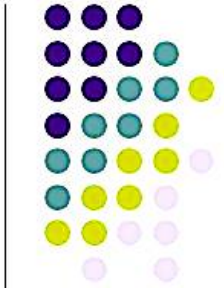
y=2000

z=3000

```
class C:public A{
    int z;
 public:
   C(){z=3000;}
   void print(){cout
<<"z="<<z<<'\n';}
};
int main(void )
{
     A a;//错误，抽象类不能产生对象
     A *pa;
     B b;
     C c;
      pa=&b;
      pa->print();
      pa=&c;
      pa->print();
}
```

扫描全能王 创建

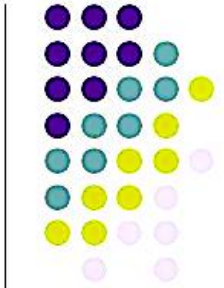
11.6.6 抽象类的设计



- 当设计较多的类时，若发现某几个类之间存在相同的成员，且这些成员具有相同的语义。则可为这些类建立一个基类，提取这些共同成员作为基类的成员。这样派生类能继承基类的成员，不再重复定义。
- 当在基类中描述成员函数时，发现一些函数的规范和语义可以确定，但具体实现不能确定。则可将这些函数修饰为纯虚函数，基类成为抽象类。

扫描全能王 创建

继承性设计：表示is a关系



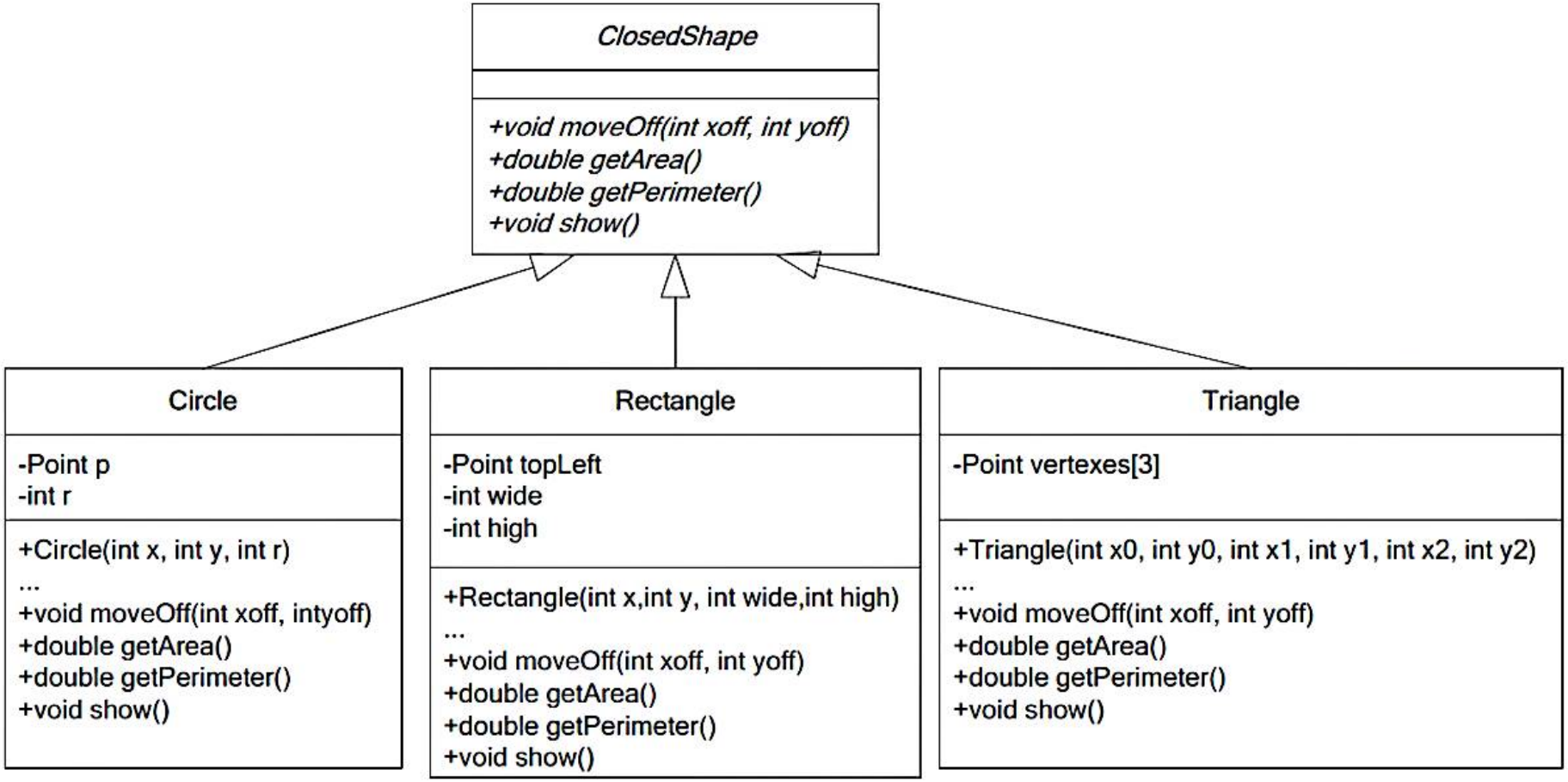
- 屏幕上的点与像素
- 人与学生、教师、进修教师
- 图形与开放图形、线段、闭合图形与矩形、三角形
- 课程与专业课程
- 教室与多媒体教室
- 文件与多媒体文件、音频文件、**MP3**文件
- 计算机与笔记本计算机
- 打印机与喷墨打印机、激光打印机、针式打印机
- 手机与智能手机、安卓手机
- **...**

错误的继承性设计

- 矩形**Rectangel**类并不是点**Point**类的派生类，因为一个矩形并不是一个点。
- 同理，圆**Circle**类也不是点**Point**类的派生类.
- 注意区别复合/成员对象之间**has a**关系与继承性**is a**关系。

扫描全能王 创建

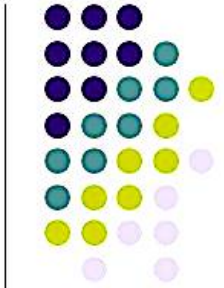
11. 用点 Point 来构造几何图形，如圆 Circle、矩形 Rectangle、三角形 Triangle 等。这些图形之间具有共同属性和行为。它们都是闭合图形，都具有确定的面积和周长。每个闭合图形都具有明确位置，也能相对移动位置。尝试建立一个抽象类 ClosedShape 作为所有闭合图形的抽象类。如下图所示。



基于抽象类建立几个通用函数设计，比如对一组图形进行移动，对一组图形计算面积并按面积排序，等等。当扩展新的派生类，比如椭圆、平行四边形，这些通用函数不用改变就能使用。

扫描全能王 创建

抽象编程的例子



在一个图形编辑器中，往往要选择几个图形作为一个整体进行相对移动：

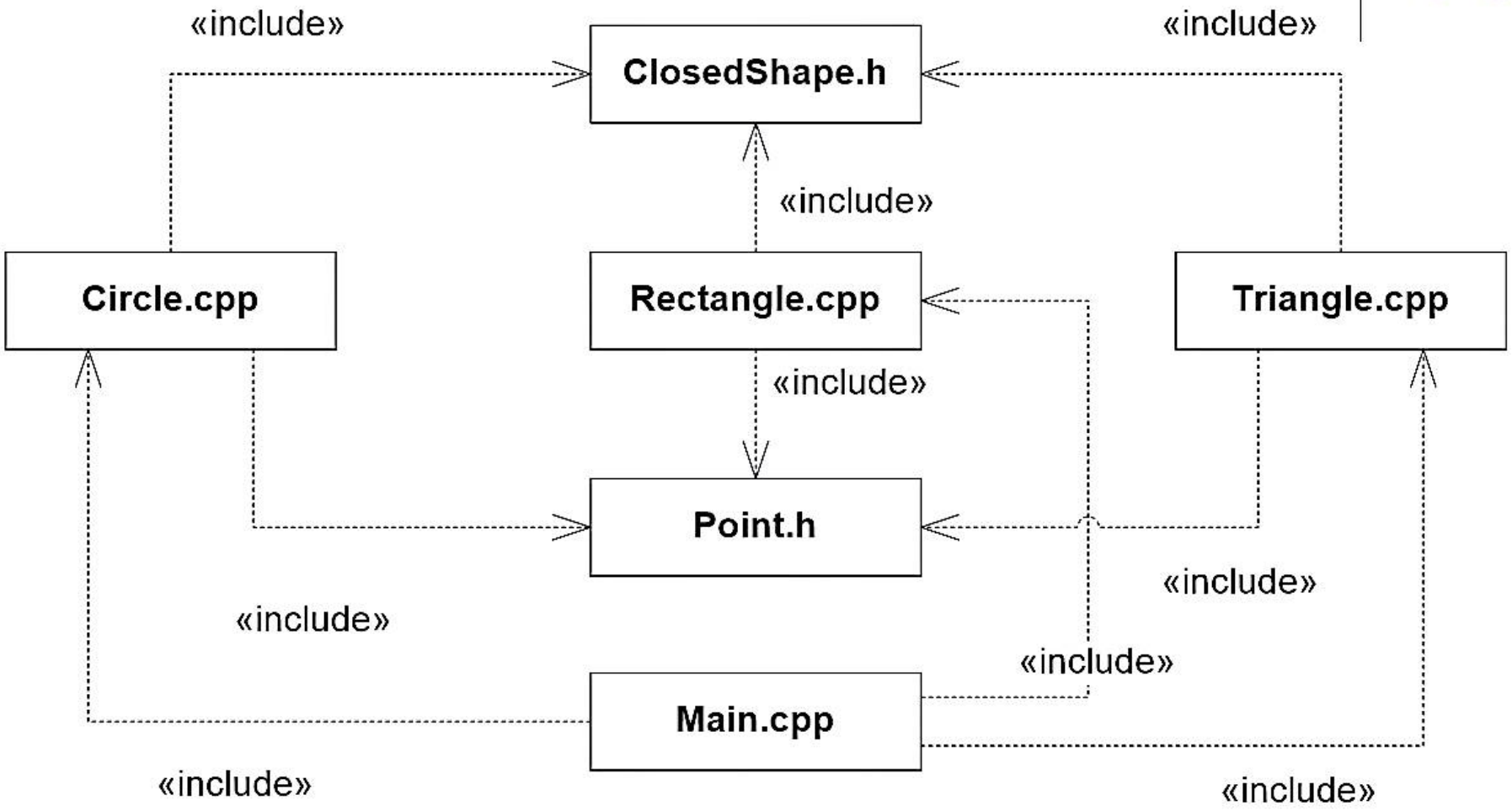
```
void moveOff(ClosedShape *selectedShapes[], int num,
   int xoff, int yoff){
   for (int i = 0; i < num; i++)
       if (selectedShapes[i] != NULL)
            selectedShapes[i]->moveOff(xoff, yoff);
}
```

要显示一个图形集合中的各个图形的当前状态：

```
void show(ClosedShape *selectedShapes[], int num){
   for (int i = 0; i < num; i++)
       if (selectedShapes[i] != NULL)
            selectedShapes[i]->show();
}
```

扫描全能王 创建

源文件结构



扫描全能王 创建